AI Assistant Coding

Assignment 19.5

Name : Varsha **HT. No :** 2303A52494 **Batch:** 50

Task 1 – Python to C++ Conversion

Task:

Translate a simple Python class into C++ using AI assistance.

Instructions:

• Prompt AI to convert a Python class representing a Student

with attributes and methods into equivalent C++ code

• Ensure proper handling of:

o Constructors

o Data types

o Access specifiers

• Compile and run both versions to verify output consistency

Expected Output:

An equivalent working C++ class translated from Python.

#PROMPT

""" convert a Python class representing a Student with attributes and methods into equivalent C++ code

Ensure proper handling of: Constructors, Data types, Access specifiers

Compile and run both versions to verify output consistency"""

#Python CODE

```python
from py_compile import main
class Student:
    def __init__(self, name: str, age: int):
        self.name = name
        self.age = age

    def display_info(self):
        print(f"Name: {self.name}, Age: {self.age}")
# Example usage
student = Student("Alice", 20)
student.display_info()
```

```cpp
# Equivalent C++ code
#include <iostream>
#include <string>
class Student {
private:
    std::string name;
    int age;
public:
    Student(std::string name, int age) : name(name), age(age) {}
    void display_info() {
        std::cout << "Name: " << name << ", Age: " << age << std::endl;
    }
};
int main() {
    Student student("Alice", 20);
    student.display_info();
    return 0;
}
```

Task 2 – Java to Python Function Conversion

Task:

Convert a Java method that checks whether a number is prime into Python.

Instructions:

- **Ask AI to translate a Java isPrime() method into Python**
- **Test the function with multiple inputs**
- **Ensure the Python version follows Pythonic syntax and logic**

Expected Output:

A correct Python function equivalent to the original Java prime-checking method.

#PROMPT

""" Translate a Java isPrime() method into Python Test the function with multiple inputs Ensure the Python version follows Pythonic syntax and logic for checking prime numbers."""

#CODE

```java
# Java code for isPrime() method
public boolean isPrime(int n) {
    if (n <= 1) {
        return false;
    }
    for (int i = 2; i <= Math.sqrt(n); i++) {
        if (n % i == 0) {
            return false;
        }
    }
    return true;
}
public static void main(String[] args) {
    int[] testNumbers = {1, 2, 3, 4, 5, 10, 11, 13, 15, 17, 19, 20};
    for (int number : testNumbers) {
        System.out.println(number + " is prime: " + isPrime(number));
    }
}
```

```python
def is_prime(n: int) -> bool:
    if n <= 1:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True
# Test the function with multiple inputs
test_numbers = [1, 2, 3, 4, 5, 10, 11, 13, 15, 17, 19, 20]
for number in test_numbers:
    print(f"{number} is prime: {is_prime(number)}")
```

Task 3 – Pseudocode to Python Implementation

Task:

Translate a given pseudocode algorithm into Python using AI.

Instructions:

• Provide AI with pseudocode for sorting numbers (e.g.,

bubble sort or selection sort)

• Ask AI to convert it into executable Python code

• Validate correctness using sample input lists

Expected Output:

A working Python program that correctly implements the

pseudocode logic.

#PROMPT

```
""" Translate the given pseudocode for sorting numbers into executable Python
code Validate correctness using sample input lists"""

# Pseudocode for sorting numbers

"""function sortNumbers(list):

   for i from 0 to length of list - 1:

      for j from 0 to length of list - i - 2:

         if list[j] > list[j + 1]:

            swap list[j] and list[j + 1]

   return list"""
```

#CODE

```python
def sort_numbers(lst):
    for i in range(len(lst)):
        for j in range(0, len(lst) - i - 1):
            if lst[j] > lst[j + 1]:
                lst[j], lst[j + 1] = lst[j + 1], lst[j]
    return lst
# Validate correctness using sample input lists
sample_list = [64, 34, 25, 12, 22, 11, 90]
sorted_list = sort_numbers(sample_list)
print("Original list:", sample_list)
print("Sorted list:", sorted_list)
```

Task 4 – SQL to Pandas Query

Task:

Translate a SQL query into a Pandas DataFrame operation in Python.

Instructions:

- **Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).**
- **Ask AI to generate equivalent Pandas code.**
- **Test the generated code on a sample DataFrame.**

Expected Output:

- **Equivalent Pandas query that retrieves the correct subset of data.**

#PROMPT

"""Translate a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000)into a Pandas DataFrame

operation in Python.Test the generated code on a sample DataFrame."""

#CODE

```python
import pandas as pd
# Sample DataFrame
df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie', 'David'],
    'salary': [60000, 45000, 70000, 55000]
})
# SQL query equivalent in Pandas
result = df[df['salary'] > 50000][['name', 'salary']]
print(result)
```

Task Description #5: Smart Hostel Room Allocation System

A hostel management system stores student room allocation details

including student ID, room number, floor, and allocation date. The

system needs to:

1. Search allocation details using student ID.

2. Sort records based on room number or allocation date.

Student Task

- **Use AI to suggest optimized algorithms.**
- **Justify the selections.**
- **Implement the solution in Python.**

#PROMPT

""" Generate a python program for A hostel management system that stores student room allocation details

including student ID, room number, floor, and allocation date. The

system needs to:

1. Search allocation details using student ID.

2. Sort records based on room number or allocation date. Recommend suitable searching and sorting algorithms.

Justify the selected algorithms."""

#CODE

```
def merge_sort(arr, key):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid], key)
    right_half = merge_sort(arr[mid:], key)

    return merge(left_half, right_half, key)
def merge(left, right, key):
    merged = []
    left_index = right_index = 0

    while left_index < len(left) and right_index < len(right):
        if key(left[left_index]) < key(right[right_index]):
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1

    # If there are remaining elements in left or right, add them to merged
    merged.extend(left[left_index:])
    merged.extend(right[right_index:])

    return merged
class Allocation:
    def _init_(self, student_id, room_number, floor, allocation_date):
        self.student_id = student_id
```

```
        self.room_number = room_number
        self.floor = floor
        self.allocation_date = allocation_date
class HostelManagementSystem:
    def _init_(self):
        self.allocations = []

    def add_allocation(self, allocation):
        self.allocations.append(allocation)

    def search_allocation_by_student_id(self, student_id):
        """
        Searches for allocation details by student ID using linear search.

        Time Complexity: O(n) - where n is the number of allocations.
        Space Complexity: O(1) - as we are using a constant amount of space for variables.
        """
        for allocation in self.allocations:
            if allocation.student_id == student_id:
                return allocation
        return None

    def sort_allocations_by_room_number(self):
        """
        Sorts records based on room number using Merge Sort.

        Time Complexity: O(n log n) - where n is the number of allocations.
        Space Complexity: O(n) - due to the temporary arrays used for merging.
        """
```

```
        self.allocations = merge_sort(self.allocations, key=lambda x: x.room_number)

    def sort_allocations_by_allocation_date(self):
        """
        Sorts records based on allocation date using Merge Sort.

        Time Complexity: O(n log n) - where n is the number of allocations.
        Space Complexity: O(n) - due to the temporary arrays used for merging.
        """
        self.allocations = merge_sort(self.allocations, key=lambda x: x.allocation_date)
# Test cases
if _name_ == "_main_":
    hms = HostelManagementSystem()
    hms.add_allocation(Allocation("S001", "101", "1st", "2023-09-01"))
    hms.add_allocation(Allocation("S002", "102", "1st", "2023-09-02"))
    hms.add_allocation(Allocation("S003", "201", "2nd", "2023-09-03"))

    # Test search
    allocation = hms.search_allocation_by_student_id("S002")
    print(f"Search result for student ID 'S002': {allocation.room_number if allocation else 'Not found'}")

    # Test sorting by room number
    hms.sort_allocations_by_room_number()
    print("Allocations sorted by room number:")
    for alloc in hms.allocations:
        print(f"Student ID: {alloc.student_id}, Room Number: {alloc.room_number}")

    # Test sorting by allocation date
    hms.sort_allocations_by_allocation_date()
```

```
print("Allocations sorted by allocation date:")
for alloc in hms.allocations:
    print(f"Student ID: {alloc.student_id}, Allocation Date: {alloc.allocation_date}")
```

#OUTPUT

```
Search result for student ID 'S002': 102
Allocations sorted by room number:
Student ID: S001, Room Number: 101
Student ID: S002, Room Number: 102
Student ID: S003, Room Number: 201
Allocations sorted by allocation date:
Student ID: S001, Allocation Date: 2023-09-01
Student ID: S002, Allocation Date: 2023-09-02
Student ID: S003, Allocation Date: 2023-09-03
```

Task Description #6: Online Movie Streaming Platform

A streaming service maintains movie records with movie ID, title,

genre, rating, and release year. The platform needs to:

1. Search movies by movie ID.

2. Sort movies based on rating or release year.

Student Task

- **Recommend searching and sorting algorithms using AI.**
- **Justify the chosen algorithms.**
- **Implement Python functions.**

#PROMPT

"""streaming service maintains movie records with movie ID, title, genre, rating, and release year.

Generate a python program that allows users to perform the following operations:

1. Search movies by movie ID.

2. Sort movies based on rating or release year. Recommend suitable searching and sorting algorithms. Justify the selected algorithms."""

#CODE

```
class Movie:
    def _init_(self, movie_id, title, genre, rating, release_year):
        self.movie_id = movie_id
        self.title = title
        self.genre = genre
        self.rating = rating
        self.release_year = release_year
class StreamingService:
    def _init_(self):
        self.movies = []

    def add_movie(self, movie):
        self.movies.append(movie)

    def search_movie_by_id(self, movie_id):
        """
        Searches for a movie by its ID using linear search.

        Time Complexity: O(n) - where n is the number of movies.
        Space Complexity: O(1) - as we are using a constant amount of space for variables.
        """
        for movie in self.movies:
            if movie.movie_id == movie_id:
                return movie
        return None

    def sort_movies_by_rating(self):
        """
        Sorts movies based on rating using Merge Sort.
```

```
        Time Complexity: O(n log n) - where n is the number of movies.
        Space Complexity: O(n) - due to the temporary arrays used for merging.
        """
        self.movies = merge_sort(self.movies, key=lambda x: x.rating)

    def sort_movies_by_release_year(self):
        """
        Sorts movies based on release year using Merge Sort.

        Time Complexity: O(n log n) - where n is the number of movies.
        Space Complexity: O(n) - due to the temporary arrays used for merging.
        """
        self.movies = merge_sort(self.movies, key=lambda x: x.release_year)
def merge_sort(arr, key):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid], key)
    right_half = merge_sort(arr[mid:], key)

    return merge(left_half, right_half, key)
def merge(left, right, key):
    merged = []
    left_index = right_index = 0

    while left_index < len(left) and right_index < len(right):
        if key(left[left_index]) < key(right[right_index]):
```

```
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1

    # If there are remaining elements in left or right, add them to merged
    merged.extend(left[left_index:])
    merged.extend(right[right_index:])

    return merged
# Test cases
if _name_ == "_main_":
    streaming_service = StreamingService()
    streaming_service.add_movie(Movie(1, "Inception", "Sci-Fi", 8.8, 2010))
    streaming_service.add_movie(Movie(2, "The Matrix", "Sci-Fi", 8.7, 1999))
    streaming_service.add_movie(Movie(3, "Interstellar", "Sci-Fi", 8.6, 2014))
    streaming_service.add_movie(Movie(4, "The Godfather", "Crime", 9.2, 1972))

    # Search for a movie by ID
    movie = streaming_service.search_movie_by_id(2)
    if movie:
        print(f"Movie found: {movie.title} ({movie.release_year}) - Rating: {movie.rating}")
    else:
        print("Movie not found.")

    # Sort movies by rating
    streaming_service.sort_movies_by_rating()
    print("\nMovies sorted by rating:")
```

```
for movie in streaming_service.movies:
    print(f"{movie.title} ({movie.release_year}) - Rating: {movie.rating}")


# Sort movies by release year
streaming_service.sort_movies_by_release_year()
print("\nMovies sorted by release year:")
for movie in streaming_service.movies:
    print(f"{movie.title} ({movie.release_year}) - Rating: {movie.rating}")
```

#OUTPUT

```
Movie found: The Matrix (1999) - Rating: 8.7

Movies sorted by rating:
Interstellar (2014) - Rating: 8.6
The Matrix (1999) - Rating: 8.7
Inception (2010) - Rating: 8.8
The Godfather (1972) - Rating: 9.2

Movies sorted by release year:
The Godfather (1972) - Rating: 9.2
The Matrix (1999) - Rating: 8.7
Inception (2010) - Rating: 8.8
Interstellar (2014) - Rating: 8.6
```

Task Description #7: Smart Agriculture Crop Monitoring System

An agriculture monitoring system stores crop data with crop ID, crop name, soil moisture level, temperature, and yield estimate. Farmers need to:

1. Search crop details using crop ID.

2. Sort crops based on moisture level or yield estimate.

Student Task

- **Use AI-assisted reasoning to select algorithms.**
- **Justify algorithm suitability.**
- **Implement searching and sorting in Python.**

#PROMPT

"""An agriculture monitoring system stores crop data with crop ID, crop name, soil moisture level, temperature, and yield estimate.

Generate a python program that allows users to perform the following operations:

1. Search crop details using crop ID.

2. Sort crops based on moisture level or yield estimate. Recommend suitable searching and sorting algorithms. Justify the selected algorithms."""

#CODE

```
class Crop:
    def __init__(self, crop_id, crop_name, soil_moisture_level, temperature, yield_estimate):
        self.crop_id = crop_id
        self.crop_name = crop_name
        self.soil_moisture_level = soil_moisture_level
        self.temperature = temperature
        self.yield_estimate = yield_estimate
class Agriculture:
    def __init__(self):
        self.crops = []

    def add_crop(self, crop):
        self.crops.append(crop)

    def search_crop_by_id(self, crop_id):
        """
        Searches for crop details by crop ID using linear search.

        Time Complexity: O(n) - where n is the number of crops.
        Space Complexity: O(1) - as we are using a constant amount of space for variables.
```

```
        """
        for crop in self.crops:
            if crop.crop_id == crop_id:
                return crop
        return None

    def sort_crops_by_moisture_level(self):
        """
        Sorts crops based on soil moisture level using Merge Sort.

        Time Complexity: O(n log n) - where n is the number of crops.
        Space Complexity: O(n) - due to the temporary arrays used for merging.
        """
        self.crops = merge_sort(self.crops, key=lambda x: x.soil_moisture_level)

    def sort_crops_by_yield_estimate(self):
        """
        Sorts crops based on yield estimate using Merge Sort.

        Time Complexity: O(n log n) - where n is the number of crops.
        Space Complexity: O(n) - due to the temporary arrays used for merging.
        """
        self.crops = merge_sort(self.crops, key=lambda x: x.yield_estimate)
def merge_sort(arr, key):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid], key)
```

```
    right_half = merge_sort(arr[mid:], key)

    return merge(left_half, right_half, key)
def merge(left, right, key):
    merged = []
    left_index = right_index = 0

    while left_index < len(left) and right_index < len(right):
        if key(left[left_index]) < key(right[right_index]):
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1

    # If there are remaining elements in left or right, add them to merged
    merged.extend(left[left_index:])
    merged.extend(right[right_index:])

    return merged
# Justification for selected algorithms:
# 1. Linear Search: We use linear search for searching crop details by crop ID because it is simple to implement and works well for small datasets. Since the number of crops may not be very large, linear search is efficient enough for this use case.
# 2. Merge Sort: We use merge sort for sorting crops based on soil moisture level and yield estimate because it is a stable sorting algorithm with a time complexity of O(n log n) in the worst case. Merge sort is efficient for sorting large datasets and can handle different types of data, making it suitable for our crop data. Additionally, merge sort does not require additional space for in-place sorting, which can be beneficial when dealing with larger datasets.
#Test cases
if __name__ == "__main__":
```

```
agriculture_system = Agriculture()

# Adding sample crops
agriculture_system.add_crop(Crop(1, "Wheat", 30, 25, 100))
agriculture_system.add_crop(Crop(2, "Corn", 40, 28, 150))
agriculture_system.add_crop(Crop(3, "Rice", 20, 22, 120))

# Searching for a crop by ID
crop = agriculture_system.search_crop_by_id(2)
if crop:
    print(f"Crop found: {crop.crop_name}, Soil Moisture Level: {crop.soil_moisture_level}, Yield Estimate: {crop.yield_estimate}")
else:
    print("Crop not found.")

# Sorting crops by soil moisture level
agriculture_system.sort_crops_by_moisture_level()
print("Crops sorted by soil moisture level:")
for crop in agriculture_system.crops:
    print(f"{crop.crop_name}: {crop.soil_moisture_level}")

# Sorting crops by yield estimate
agriculture_system.sort_crops_by_yield_estimate()
print("Crops sorted by yield estimate:")
for crop in agriculture_system.crops:
    print(f"{crop.crop_name}: {crop.yield_estimate}")
```

#OUTPUT

```
Crop found: Corn, Soil Moisture Level: 40, Yield Estimate: 150
Crops sorted by soil moisture level:
Rice: 20
Wheat: 30
Corn: 40
Crops sorted by yield estimate:
Wheat: 100
Rice: 120
Corn: 150
```

Task Description #8: Airport Flight Management System

An airport system stores flight information including flight ID,

airline name, departure time, arrival time, and status. The system

must:

1. Search flight details using flight ID.

2. Sort flights based on departure time or arrival time.

Student Task

• Use AI to recommend algorithms.

• Justify the algorithm selection.

• Implement searching and sorting logic in Python.

#PROMPT

""" An airport system stores flight information including flight ID, airline name, departure time, arrival time, and status. Generate a python program that allows users to perform the following operations:

1. Search flight details using flight ID.

2. Sort flights based on departure time or arrival time"""”

#CODE

```
class Flight:
    def _init_(self, flight_id, airline_name, departure_time, arrival_time, status):
        self.flight_id = flight_id
        self.airline_name = airline_name
        self.departure_time = departure_time
        self.arrival_time = arrival_time
        self.status = status
```

```
class AirportSystem:
    def _init_(self):
        self.flights = []

    def add_flight(self, flight):
        self.flights.append(flight)

    def search_flight_by_id(self, flight_id):
        """
        Searches for flight details by flight ID using linear search.

        Time Complexity: O(n) - where n is the number of flights.
        Space Complexity: O(1) - as we are using a constant amount of space for variables.
        """
        for flight in self.flights:
            if flight.flight_id == flight_id:
                return flight
        return None

    def sort_flights_by_departure_time(self):
        """
        Sorts flights based on departure time using Merge Sort.

        Time Complexity: O(n log n) - where n is the number of flights.
        Space Complexity: O(n) - due to the temporary arrays used for merging.
        """
        self.flights = merge_sort(self.flights, key=lambda x: x.departure_time)

    def sort_flights_by_arrival_time(self):
```

```
        """
        Sorts flights based on arrival time using Merge Sort.

        Time Complexity: O(n log n) - where n is the number of flights.
        Space Complexity: O(n) - due to the temporary arrays used for merging.
        """
        self.flights = merge_sort(self.flights, key=lambda x: x.arrival_time)
def merge_sort(arr, key):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid], key)
    right_half = merge_sort(arr[mid:], key)

    return merge(left_half, right_half, key)
def merge(left, right, key):
    merged = []
    left_index = right_index = 0

    while left_index < len(left) and right_index < len(right):
        if key(left[left_index]) <= key(right[right_index]):
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1

    merged.extend(left[left_index:])
```

```
    merged.extend(right[right_index:])

    return merged
# Example usage
if _name_ == "_main_":
    airport_system = AirportSystem()

    # Adding some flights
    airport_system.add_flight(Flight("FL001", "Airline A", "08:00", "10:00", "On Time"))
    airport_system.add_flight(Flight("FL002", "Airline B", "09:00", "11:00", "Delayed"))
    airport_system.add_flight(Flight("FL003", "Airline C", "07:30", "09:30", "On Time"))

    # Searching for a flight by ID
    flight = airport_system.search_flight_by_id("FL002")
    if flight:
        print(f"Flight found: {flight.airline_name}, Departure: {flight.departure_time}, Arrival:
{flight.arrival_time}, Status: {flight.status}")
    else:
        print("Flight not found.")

    # Sorting flights by departure time
    airport_system.sort_flights_by_departure_time()
    print("\nFlights sorted by departure time:")
    for flight in airport_system.flights:
        print(f"{flight.flight_id}: {flight.departure_time}")

    # Sorting flights by arrival time
    airport_system.sort_flights_by_arrival_time()
    print("\nFlights sorted by arrival time:")
    for flight in airport_system.flights:
```

```
print(f"{flight.flight_id}: {flight.arrival_time}")
```

#OUTPUT

```
Flight found: Airline B, Departure: 09:00, Arrival: 11:00, Status: Delayed

Flights sorted by departure time:
FL003: 07:30
FL001: 08:00
FL002: 09:00

Flights sorted by arrival time:
FL003: 09:30
FL001: 10:00
FL002: 11:00
```